The Poly Language Guide

A friendly, step-by-step handbook that teaches you every Poly command, with a Rust translation for each one.

Every example in Poly **and** in Rust. Written so a kid can read it.

October 2026

Contents

The Poly Language Guide 6

Chapter 1: Meet Poly 7

What is Poly? 7

Your very first program 7

Anatomy of a Poly program 7

The same program in Rust 8

How to run this book's examples 8

Chapter 2: Comments — talking to humans 9

Chapter 3: Variables — labeled boxes 10

The golden rule: := stores, = compares 11

Types — what kind of thing a box holds 11

Changing types with as 12

Chapter 4: Output — putting words on the screen 13

The put command 13

The error, warn, and info commands 14

String interpolation — putting values inside text 15

Chapter 5: Input — the get command 17

Chapter 6: Operators — doing maths and making comparisons 20

Arithmetic 20

Mutation commands: add and sub 21

The ^ power operator 21

Comparison 22

Logical words 23

Bitwise words (for tinkerers) 24

The full operator shelf 25

Chapter 7: Making decisions — if, else if, else 26

Combining conditions 27

Chapter 8: Loops — making the computer repeat itself 28

The while loop — repeat while true 28

The infinite loop — repeat until you say stop 29

The range loop — count from A to B 30

The collection loop — visit every item 32

break and continue — the loop's steering wheel 33

One loop to rule them all — a real program 34

Chapter 9: Functions — your own reusable recipes 35

Functions that decide something 36

Chapter 10: Strings — working with text 38

Gluing strings with + ... 38

Measuring with .len() 38

Turning numbers into text with .to_string() 39

String methods you will use 39

Characters 40

Chapter 11: Lists and tuples — many values in one box 41

Lists (vectors) 41

Tuples — a tiny fixed grab bag 42

Chapter 12: Structs — making your own box shapes 44

Giving a struct jobs with impl 45

Chapter 13: Enums and match — choices with style 47

Enums 47

Variants that carry data 48

The match command — pick the right branch 49

Matching on numbers and text 50

match as an expression — the branch gives you a value 50

Binding values out of a variant 51

Chapter 14: Errors — Result, Ok, Error, and try 53

Why not just crash? 53

Handling a Result with match . . . 53
Your own error types . . . 54
try — pass the problem upstairs (without all the matching) . . . 55

Chapter 15: Closures — functions you can carry around . . . 57
Higher-order functions — functions that take functions . . . 58

Chapter 16: Traits and generics — same recipe, many ingredients . . . 60
Traits (the contract) . . . 60
Generics () — one recipe, any ingredient . . . 61

Chapter 17: Macros, modules, and other organizers . . . 63
Macros — copy-and-paste power . . . 63
Modules — rooms in your house . . . 64
use — import someone else's toolbox . . . 65
type — giving a type a nickname . . . 65
pub — the visibility keyword . . . 66

Chapter 18: Foreign blocks — slipping in real Rust (or C, asm, JS) . . . 67
The rules of foreign blocks . . . 68
Telling Poly about a foreign function (optional extern) . . . 68
Bringing in outside crates with dep . . . 68
Which block for which target? . . . 69

Chapter 19: Async and spawn — waiting without waiting . . . 70

Chapter 20: The poly command — running your programs . . . 72
The everyday commands . . . 72
Choosing a target . . . 72
Looking at what Poly made . . . 72
Checking without building . . . 72
Peeking under the hood . . . 73
Bigger tools . . . 73
Flag order does not matter . . . 73
A safety warning, seriously . . . 73

Chapter 21: When your program grumbles — common mistakes 74

== is rejected 74

A block that never ends 74

var x: i32 := 5 is wrong 74

Retired syntax the compiler will nag about 74

Reserved words — names you cannot use 75

Depth limits 75

A foreign block inside a function 75

Type complaints 75

Appendix A: The type menu 76

Appendix B: The operator shelf 77

Appendix C: Every command at a glance 78

Appendix D: Poly to Rust — the big cheat sheet 80

Glossary 82

Where to go next 83

The Poly Language Guide

A friendly, step-by-step handbook that teaches you every Poly command, with a Rust translation for each one.

Welcome, young programmer! This book explains the **Poly** programming language, one command at a time. For every command you will get:

1. A plain-English explanation of what it does.
2. Its **syntax** (the exact shape you type).
3. A small **Poly** code snippet showing how to use it.
4. The **equivalent Rust** code, so you can see what Poly turns into.

You do not need to know Rust to read this book. You do not need to be an adult either. If you can read and type, you can learn Poly.

Chapter 1: Meet Poly

What is Poly?

Poly is a small programming language with big, friendly words. Instead of curly braces `{ }` everywhere, Poly uses words like `put`, `loop`, `end fn`, and `if`.

Poly is also a **transpiler**. That is a fancy word for "code translator." Poly reads your `.poly` file, translates it into another language (Rust, C, assembly, JavaScript, or Goose), and then the real compiler for that language builds your program.

```
  my_program.poly  --(Poly)-->  my_program.rs  --(rustc)-->  executable
```

Why is that cool? Because you write the simple, easy part, and Poly handles the boring plumbing for you. When you need something powerful, you can slip real Rust (or C, or JavaScript, or Goose) straight into your file.

Your very first program

Every Poly program lives in a file whose name ends in `.poly`. Type this into a new file called `hello.poly`:

```
fn main()
    put "Hello, World!"
end fn
```

Save it. Then run it:

```
poly hello.poly
```

You will see something like this:

```
Generated and built hello.poly -> rust_output/hello/target/debug/hello
```

Poly translated your program to Rust and built an executable. Run the executable to see your message:

```
./rust_output/hello/target/debug/hello
```

Output:

```
Hello, World!
```

Anatomy of a Poly program

Let's cut open `hello.poly` like a science project and look at the parts:

Part	What it means
`fn main()`	"Start the program here." `main` is the special first function.
`put "Hello, World!"`	A command: print this text on the screen.
`end fn`	"That is the end of the function." Every `fn` needs one.

Three tiny rules to remember:

- Every program starts with `fn main()` and ends with the matching `end fn`.
  Commands go **inside** the function, indented (spaces) so they are easy to read.
- `put` always prints a new line after what it says.

The same program in Rust

Here is what Poly builds for you behind the scenes (roughly):

```
fn main() {
    println!("{}", "Hello, World!");
}
```

Same idea, more punctuation. Poly just skips the braces and the semicolons.

How to run this book's examples

Every complete example in this book can be saved as a `.poly` file and run with `poly yourfile.poly`. Chapter 20 explains every command-line flag in detail. The short version:

- `poly program.poly` — translate and build your program.
- `poly --check program.poly` — check your program without building.
- `poly --emit-rust program.poly` — print the Rust that Poly generated.

Chapter 2: Comments — talking to humans

What it does: A comment is a note for people. The computer reads your program, skips comments completely, and never gets confused by them.

Syntax:

```
# A Poly comment (the usual style): runs to the end of the line.
// A C++-style comment also runs to the end of the line.
/* A block comment can
   span several lines. */
```

The same shape in Rust:

```
// A Rust comment (the only style): runs to the end of the line.
/* A block comment can
   span several lines. */
```

How to use it:

```
# This program greets my best friend.
fn main()
    var name ustring := "Sam"    # A comment can trail a line of code
    put "Hello, " + name + "!"
end fn
```

Output:

```
Hello, Sam!
```

The same thing in Rust:

```
// This program greets my best friend.
fn main() {
    let mut name: String = String::from("Sam"); // trails the line
    println!("Hello, {}!", name);
}
```

Good to know:

- Use # for Poly comments. It is the house style.

  One rule to watch: a line that starts with `#rust`, `#c`, `#asm`, or `#js` is **not** a comment — it opens a foreign code block (Chapter 18).

  Comments are your future self's best friend. Write what the code is *for*, not what it *does* line by line.

Chapter 3: Variables — labeled boxes

What it does: A variable is a labeled box that holds a value. You put something in the box, give the box a name, and later you can look inside or change what is in it.

Syntax:

```
var name Type := value     # a box with a known type (mutable)
var name := value          # a box where Poly guesses the type (mutable)
let name: Type := value    # a box you may NEVER change (immutable)
const NAME := value        # a number fixed for the whole program
```

The same shape in Rust:

```
let mut name: Type = value; // a box with a known type (mutable)
let mut name = value;       // a box where Rust guesses the type (mutable)
let name: Type = value;     // a box you may NEVER change (immutable)
const NAME: Type = value;   // a value fixed for the whole program
```

How to use it:

```
fn main()
    var score i32 := 10          # a box called score, holds a whole number
    var nickname := "Speedy"     # Poly figures out this one is a string
    let player: ustring := "Ada" # player can never be re-assigned
    const MAX_LIVES := 3         # a constant: 3 forever and ever
    score := score + 1           # := puts a NEW value in the box
    put score
    put nickname
    put player
    put MAX_LIVES
end fn
```

Output:

```
11
Speedy
Ada
3
```

The same thing in Rust:

```
fn main() {
    let mut score: i32 = 10;          // mutable, so Rust needs `mut`
    let mut nickname = String::from("Speedy");
    let player: String = String::from("Ada"); // never re-assigned: no `mut`
    const MAX_LIVES: i32 = 3;
    score = score + 1;
    println!("{}", score);
    println!("{}", nickname);
    println!("{}", player);
```

```
    println!("{}", MAX_LIVES);
}
```

The golden rule: := stores, = compares

This is the number one thing to remember in all of Poly:

You see	It means	Example
`:=`	"store this value"	`score := score + 1`
`=`	"are these two values equal?"	`if score = 10`

Rust flips it around: Rust uses = to store and == to compare. In Poly, == is rejected — the compiler will kindly tell you to use =.

Careful! There is no += in Poly. Write the whole thing out:

```
count := count + 1     # right
count += 1             # wrong: retired syntax, rejected
```

The same shape in Rust:

```
count = count + 1; // right: in Rust `=` is the store sign
count += 1;        // fine in Rust — but remember: no `+=` in Poly
```

Types — what kind of thing a box holds

A type says what a box can hold: whole numbers, decimals, words, true/false. Poly can often guess ("infer") the type, but you can always say it yourself.

```
fn main()
    var age i32 := 9            # a 32-bit whole number
    var temperature f64 := 36.6 # a decimal number
    var is_happy bool := true   # true or false
    var first_letter char := unicode 'A'   # one character
    var name ustring := "Riley" # a Unicode text string
    put age
    put temperature
    put is_happy
    put first_letter
    put name
end fn
```

Output:

```
9
36.6
true
A
```

```
Riley
```

The same thing in Rust:

```
fn main() {
    let age: i32 = 9;
    let temperature: f64 = 36.6;
    let is_happy: bool = true;
    let first_letter: char = 'A';
    let name: String = String::from("Riley");
    println!("{}", age);
    println!("{}", temperature);
    println!("{}", is_happy);
    println!("{}", first_letter);
    println!("{}", name);
}
```

The full type menu is in Appendix A. The ones you will use every day:

- `i32` — whole numbers (positive, negative, zero).
- `f64` — decimal numbers.
- `bool` — `true` or `false`.
- `ustring` — text.
- `char` — one single character, written like `'A'`.

Changing types with as

Sometimes two boxes hold different kinds of numbers and Poly will not let you mix them (that is a feature: it catches bugs). Convert on purpose with `as`:

```
var big u64 := 7
var small i32 := big as i32   # "pretend this u64 is an i32"
```

The same shape in Rust:

```
let big: u64 = 7;
let small: i32 = big as i32; // "pretend this u64 is an i32"
```

The same thing in Rust:

```
let big: u64 = 7;
let small: i32 = big as i32;
```

Chapter 4: Output — putting words on the screen

The `put` command

What it does: `put` prints a value to the screen (standard output), then adds a newline — the "Enter key" — so the next thing you print starts on a fresh line.

Syntax:

```
put expression
put expression to "file.txt"             # write to a file (overwrites!)
put expression to "file.txt" -append     # add to the end of a file
put expression to "file.txt" -prepend    # add to the start of a file
```

The same shape in Rust:

```
println!("{}", expression);          // print + newline (stdout)
std::fs::write("file.txt", text)?;   // write to a file (overwrites!)
// -append: OpenOptions::new().append(true).open("file.txt")
// -prepend: read_to_string("file.txt") first, insert the new line, write back
```

How to use it:

```
fn main()
    put "Hello, World!"        # text (a string)
    put 42                     # a whole number
    put 3.14                   # a decimal number
    put true                   # a true/false value
    put "I am " + 10 + " years old"   # + glues things together
end fn
```

Output:

```
Hello, World!
42
3.14
true
I am 10 years old
```

The same thing in Rust:

```
fn main() {
    println!("{}", "Hello, World!");
    println!("{}", 42);
    println!("{}", 3.14);
    println!("{}", true);
    println!("I am {} years old", 10);
}
```

Putting text into a file

```
fn main()
    # First put with `to` creates (or re-writes) the file
    put "Line number 1" to "diary.txt"
    # -append keeps what is already there and adds to the end
    put "Line number 2" to "diary.txt" -append
    put "Check diary.txt!"
end fn
```

Output:

```
Check diary.txt!
```

The same thing in Rust:

```
use std::io::Write;
fn main() {
    std::fs::write("diary.txt", "Line number 1\n").unwrap();
    let mut file = std::fs::OpenOptions::new()
        .append(true)
        .open("diary.txt")
        .unwrap();
    writeln!(file, "Line number 2").unwrap();
    println!("Check diary.txt!");
}
```

> ***Tip:*** `put ... to "file"` ***overwrites*** *the file every time. Use* `-append` *when you want to keep the old contents (adding to the end), or* `-prepend` *to keep them with the new line in front.*

The `error`, `warn`, and `info` commands

What it does: These three print a message to **standard error** (a second output pipe, where programs report problems) with a tag in front. Use them to say how serious your message is.

Syntax:

```
error expression    # printed with [ERROR] — something broke
warn expression     # printed with [WARN]  — something looks suspicious
info expression     # printed with [INFO]  — a note for developers
```

The same shape in Rust:

```
eprintln!("[ERROR] {}", expression); // something broke
eprintln!("[WARN] {}", expression);  // something looks suspicious
eprintln!("[INFO] {}", expression);  // a note for developers
```

How to use it:

```
fn main()
```

```
    info "Starting the game..."
    warn "That level has not been tested yet"
    error "Save file is missing!"
    put "Program finished anyway"
end fn
```

Output (the first three lines go to stderr):

```
[INFO] Starting the game...
[WARN] That level has not been tested yet
[ERROR] Save file is missing!
Program finished anyway
```

The same thing in Rust:

```
fn main() {
    eprintln!("[INFO] {}", "Starting the game...");
    eprintln!("[WARN] {}", "That level has not been tested yet");
    eprintln!("[ERROR] {}", "Save file is missing!");
    println!("Program finished anyway");
}
```

Good to know:

- `put` prints to stdout (normal output). The other three print to stderr.

  That matters when you send output into a file: `./mygame > out.txt` captures stdout only, so errors still show on your screen.

String interpolation — putting values inside text

What it does: Wrap a value in braces `{ }` inside a string and Poly slips the value in for you. No + glue needed.

Syntax:

```
put "text {variable} more text"
```

The same shape in Rust:

```
println!("text {} more text", variable);
```

How to use it:

```
fn main()
    var name ustring := "Zoe"
    var age i32 := 11
    put "My name is {name} and I am {age} years old"
end fn
```

Output:

```
My name is Zoe and I am 11 years old
```

The same thing in Rust:

```
fn main() {
    let name: String = String::from("Zoe");
    let age: i32 = 11;
    println!("My name is {} and I am {} years old", name, age);
}
```

Interpolation works on every target (Rust, C, assembly, JavaScript, and Goose).

Chapter 5: Input — the `get` command

What it does: `get` reads what the user types and hands it to your program. It waits for the Enter key.

Syntax:

```
var x := get                                  # read a line of text
var x := get unicode "Prompt: "               # show a prompt first
var x := get --as i32             # convert the typed text to a number
var x := get --default unicode "fallback"     # value used when input is empty
var x := get --mask unicode "*"               # hide the typing (passwords!)
var x := get --until unicode ","              # stop at a delimiter
var x := get --timeout 3000                   # give up after 3000 ms
var x := get from "file.txt"                  # read from a file instead
var x bytes := get from "data.bin" --bytes 8  # read exactly 8 bytes
```

The same shape in Rust:

```
// Rust has no one-command input, so each flag is spelled out by hand:
let mut line = String::new();
std::io::stdin().read_line(&mut line)?;          // read a line of text
// --as i32:        line.trim().parse::<i32>()?
// --default:       if line.is_empty() { fallback } else { line }
// --mask/--until/--timeout: extra code (or a crate) — Poly does it for you
// from "file.txt": std::fs::read("file.txt")? / std::fs::read_dir(...)?
```

How to use it — the basic form:

```
fn main()
    put "What is your name?"
    var name ustring := get     # get waits for the user to press Enter
    put "Hello, " + name + "!"
end fn
```

Output (type `Sam` and press Enter):

```
What is your name?
Hello, Sam!
```

How to use it — numbers, defaults, and masks:

```
fn main()
    put "How old are you?"
    var age i32 := get --as i32           # turns typed text into a number
    put "Next year you will be " + (age + 1)
    put "Favorite color (or press Enter): "
    var color ustring := get --default unicode "blue"   # if input is empty
    put "Color: " + color
    put "Password: "
    var password ustring := get --mask unicode "*"      # typed keys show as *
```

```
    put "Password length: " + password.len()
end fn
```

Output (you type 9, press Enter for the color, then type `hunter2`):

```
How old are you?
Next year you will be 10
Favorite color (or press Enter):
Color: blue
Password:
Password length: 7
```

The same thing in Rust:

```
use std::io::{self, Write};
fn read_line() -> String {
    let mut line = String::new();
    io::stdin().read_line(&mut line).unwrap();
    line.trim().to_string()
}
fn main() {
    println!("How old are you?");
    let age: i32 = read_line().parse().unwrap();
    println!("Next year you will be {}", age + 1);
    println!("Favorite color (or press Enter): ");
    let input = read_line();
    let color = if input.is_empty() { "blue".to_string() } else { input };
    println!("Color: {}", color);
    print!("Password: ");
    io::stdout().flush().unwrap();
    let password = read_line(); // masking the terminal is OS-specific in Rust
    println!("Password length: {}", password.len());
}
```

Notice how much longer the Rust is — that is Poly's whole job: doing the boring parts for you.

Every `get` flag at a glance:

Flag	What it does
`--as TYPE`	Convert the input to a type, like `--as i32`.
`--default VALUE`	Use this value when the user submits an empty line.
`--mask CHAR`	Echo a star (or any character) instead of typed keys.
`--until DELIM`	Read only up to the delimiter (the delimiter is left out).
`--timeout MS`	Give up after this many milliseconds.
`from "file"`	Read from a file instead of the keyboard.
`--bytes N`	Read exactly N bytes from a file.

Good to know:

`get` with flags is a Rust-target feature. The C target supports plain `get` (with an optional prompt); the other flags need a helper in a `#c` block (Chapter 18).

- Validate what the user types by asking again in a loop:

```
var age i32 := 0
var done bool := false
while not done
    var input i32 := get --as i32
    if input >= 1 and input <= 150
        age := input
        done := true
    else
        put "Please type an age between 1 and 150"
    end if
end while
```

The same shape in Rust:

```
let mut age: i32 = 0;
let mut done = false;
while !done {
    let mut line = String::new();
    std::io::stdin().read_line(&mut line)?;
    let input: i32 = line.trim().parse().unwrap_or(0);
    if input >= 1 && input <= 150 {
        age = input;
        done = true;
    } else {
        println!("Please type an age between 1 and 150");
    }
}
```

Chapter 6: Operators — doing maths and making comparisons

What it does: Operators are the little symbols that calculate or compare values. Poly has four families: arithmetic, comparison, logical, and bitwise.

Arithmetic

Poly	Meaning	Example	Rust equivalent
`+`	add	`2 + 3`	`2 + 3`
`-`	subtract	`10 - 4`	`10 - 4`
`*`	multiply	`6 * 7`	`6 * 7`
`/`	divide	`9 / 3`	`9 / 3`
`mod`	remainder	`10 mod 3`	`10 % 3`

How to use it:

```
fn main()
    var a i32 := 10
    var b i32 := 3
    put a + b     # 13
    put a - b     # 7
    put a * b     # 30
    put a / b     # 3  (whole numbers lose the fraction)
    put a mod b   # 1  (10 divided by 3 is 3 remainder 1)
end fn
```

Output:

```
13
7
30
3
1
```

The same thing in Rust:

```
fn main() {
    let a: i32 = 10;
    let b: i32 = 3;
    println!("{}", a + b);
    println!("{}", a - b);
    println!("{}", a * b);
    println!("{}", a / b);
    println!("{}", a % 3);
}
```

> ***Remember:*** *Poly spells the remainder operator as the word* `mod`*. The symbol* `%` *is retired. But* `^` ***is*** *"to the power of":* `x ^ 2` *is* `x` *squared. Need it? It is built in — see the next section.*

Mutation commands: add and sub

What they do: `add x, n` adds `n` to `x`, and `sub x, n` subtracts it. Each is a shortcut that lowers to an ordinary assignment — `x := x + n` and `x := x - n`. The target must be a plain variable, and **the comma is required**: `add x n` (no comma) is rejected as retired syntax.

How to use it:

```
fn main()
    var total i32 := 10
    add total, 5      # total is now 15  (total := total + 5)
    sub total, 3      # total is now 12  (total := total - 3)
    put total         # 12
end fn
```

Output:

```
12
```

The same thing in Rust:

```
fn main() {
    let mut total: i32 = 10;
    total += 5;   // add total, 5
    total -= 3;   // sub total, 3
    println!("{}", total);
}
```

> ***Remember:*** `add` *and* `sub` *are live commands only in this comma form; the old bare spellings* `add x n` / `sub x n` *are retired (see the retired-syntax table at the end of the book).* `inc x` *and* `dec x` *are live too — they take no value and mean* `x := x + 1` / `x := x - 1`*.*

The ^ power operator

What it does: `a ^ b` raises `a` to the power `b` (exponentiation). It binds **tighter** than `*` and `/`, and it groups to the **right**, just like in maths.

Syntax:

```
base ^ exponent        # 2 * 3 ^ 2 means 2 * (3 ^ 2): powers go first!
2 ^ 3 ^ 2              # right-associative: 2 ^ (3 ^ 2) = 512
```

How to use it:

```
fn main()
    var x i32 := 7
```

```
    var square i32 := x ^ 2      # 49
    var area f64 := 3.0 ^ 2.0    # floats work too: 9.0
    put square
    put area
end fn
```

Output:

```
49
9
```

The same thing in Rust:

```
fn main() {
    let x: i32 = 7;
    let square: i32 = x.wrapping_pow(2 as u32); // 49
    let area: f64 = 3.0_f64.powf(2.0);           // 9.0
    println!("{}", square);
    println!("{}", area);
}
```

Good to know:

Integer powers **wrap** on overflow on every target, like all Poly arithmetic — no surprises between languages.

Both sides must be the same kind of number (both whole numbers or both decimals). Mix them and the compiler asks for an explicit `as` cast, like every other arithmetic operator.

A negative whole-number exponent gives `0` at runtime (there is no whole number that is `2 ^ -1`). Use a float base for fractional powers: `2.0 ^ 0.5`.

Comparison

Poly	Meaning	Rust equivalent
=	equal to	==
!=	not equal to	!=
<	less than	<
>	greater than	>
<=	less than or equal	<=
>=	greater than or equal	>=

How to use it:

```
fn main()
    var score i32 := 80
    if score >= 60
        put "You passed!"
    else
```

```
        put "Try again"
    end if
    put score = 80     # true — one = compares in Poly
end fn
```

Output:

```
You passed!
true
```

The same thing in Rust:

```
fn main() {
    let score: i32 = 80;
    if score >= 60 {
        println!("You passed!");
    } else {
        println!("Try again");
    }
    println!("{}", score == 80);
}
```

Poly compares **strictly**: both sides must be the same type. `u64 = i64` is an error. Convert first with `as` when you really mean to compare across types.

Logical words

Poly	Meaning	Rust equivalent
`and`	both must be true	&&
`or`	at least one true	`\`
`not`	flips true/false	!

How to use it:

```
fn main()
    var age i32 := 12
    var has_ticket bool := true
    if age >= 12 and has_ticket
        put "Come on in!"
    end if
    if not has_ticket
        put "You need a ticket"
    end if
end fn
```

Output:

```
Come on in!
```

The same thing in Rust:

```
fn main() {
    let age: i32 = 12;
    let has_ticket: bool = true;
    if age >= 12 && has_ticket {
        println!("Come on in!");
    }
    if !has_ticket {
        println!("You need a ticket");
    }
}
```

The symbols `&&`, `||`, and `!` are retired in Poly — use the words.

Bitwise words (for tinkerers)

Bitwise operators poke at the individual bits inside a whole number.

Poly	Meaning	Rust equivalent
`a bitand b`	bit AND	`a & b`
`a bitor b`	bit OR	`a | b`
`a xor b`	bit exclusive OR	`a ^ b`
`bitnot a`	flip every bit	`!a`
`a shift left n`	slide bits left	`a << n`
`a shift right n`	slide bits right	`a >> n`

How to use it:

```
fn main()
    var flags i32 := 0b1010
    put flags bitand 0b1100    # 0b1000 (as a number: 8)
    put flags xor 0b1111       # 0b0101 (as a number: 5)
    put flags shift left 1     # slide every bit one spot left
end fn
```

Output:

```
8
5
20
```

The same thing in Rust:

```
fn main() {
    let flags: i32 = 0b1010;
    println!("{}", flags & 0b1100);
    println!("{}", flags ^ 0b1111);
```

```
    println!("{}", flags << 1);
}
```

<< and >> are still allowed in Poly; the symbol spellings of the other bitwise operators are rejected.

The full operator shelf

```
Arithmetic:   +   -   *   /   mod
Comparison:   =   !=  <   >   <=  >=
Logical:      and  or  not
Bitwise:      bitand  bitor  xor  bitnot  shift left  shift right  <<  >>
Store value:  :=
Ranges:       ..  ..=          (used by loop, Chapter 8)
Path:         ::              (used by enums, Chapter 13)
Cast:         as              (change a value's type)
```

Chapter 7: Making decisions — `if`, `else if`, `else`

What it does: An `if` block runs its commands only when a condition is true. `else if` checks a second condition, and `else` catches everything else. One `end if` closes the whole chain.

Syntax:

```
if condition
    # runs when condition is true
else if second condition
    # runs when the first was false and this one is true
else
    # runs when nothing above matched
end if
```

The same shape in Rust:

```
if condition {
    // runs when condition is true
} else if second_condition {
    // runs when the first was false and this one is true
} else {
    // runs when nothing above matched
}
```

How to use it:

```
fn main()
    var score i32 := 75
    if score >= 90
        put "Grade: A"
    else if score >= 80
        put "Grade: B"
    else if score >= 70
        put "Grade: C"
    else
        put "Keep practicing!"
    end if
end fn
```

Output:

```
Grade: C
```

The same thing in Rust:

```
fn main() {
    let score: i32 = 75;
    if score >= 90 {
```

```
        println!("Grade: A");
    } else if score >= 80 {
        println!("Grade: B");
    } else if score >= 70 {
        println!("Grade: C");
    } else {
        println!("Keep practicing!");
    }
}
```

Rules of the road:

One `end if` for the whole chain. Every `else if` and the final `else` share it.

Do not nest a fresh `if` directly inside an `else` — fold it into the chain as another `else if`.

- Indent the bodies. Poly does not care, but humans do.

  You may write `if condition,` with a comma (old style), but new code should leave the comma out.

Combining conditions

```
fn main()
    var age i32 := 10
    var rainy bool := false
    if age >= 8 and not rainy
        put "Go play outside!"
    else if rainy and age >= 5
        put "Play inside instead"
    else
        put "Too young for the park"
    end if
end fn
```

Output:

```
Go play outside!
```

The same thing in Rust:

```
fn main() {
    let age: i32 = 10;
    let rainy: bool = false;
    if age >= 8 && !rainy {
        println!("Go play outside!");
    } else if rainy && age >= 5 {
        println!("Play inside instead");
    } else {
        println!("Too young for the park");
    }
}
```

Chapter 8: Loops — making the computer repeat itself

A loop runs the same commands over and over. Poly gives you four loop flavors: `while`, infinite `loop`, range `loop`, and collection `loop`.

The `while` loop — repeat while true

What it does: Keep doing the body **as long as** the condition stays true. When it turns false, move on.

Syntax:

```
while condition
    # things to repeat
    break        # jump out early
    continue     # skip to the next round
end while
```

The same shape in Rust:

```
while condition {
    // things to repeat
    break;     // jump out early
    continue;  // skip to the next round
}
```

How to use it:

```
fn main()
    var countdown i32 := 3
    while countdown > 0
        put countdown
        countdown := countdown - 1
    end while
    put "Liftoff!"
end fn
```

Output:

```
3
2
1
Liftoff!
```

The same thing in Rust:

```
fn main() {
    let mut countdown: i32 = 3;
```

```
    while countdown > 0 {
        println!("{}", countdown);
        countdown = countdown - 1;
    }
    println!("Liftoff!");
}
```

The infinite `loop` — repeat until you say stop

What it does: Runs forever (forever!) until a `break` jumps out.

Syntax:

```
loop
    # runs again and again
    break     # the only way out
end loop
```

The same shape in Rust:

```
loop {
    // runs again and again
    break; // the only way out
}
```

How to use it:

```
fn main()
    var tickets i32 := 2
    loop
        if tickets = 0
            break            # leave the loop right now
        end if
        put "Admit one"
        tickets := tickets - 1
    end loop
    put "Sold out"
end fn
```

Output:

```
Admit one
Admit one
Sold out
```

The same thing in Rust:

```
fn main() {
    let mut tickets: i32 = 2;
    loop {
        if tickets == 0 {
```

```
            break;
        }
        println!("Admit one");
        tickets = tickets - 1;
    }
    println!("Sold out");
}
```

The range `loop` — count from A to B

What it does: Gives your loop variable every number in a range. **Both endpoints are included**, so `0..3` means 0, 1, 2, 3.

Syntax:

```
loop variable from..to              # both ends included
loop variable from..to step amount  # skip by a custom amount
loop variable 10..1 step -1         # negative step counts DOWN
loop variable 1..3, 7, 19..20       # several ranges and values at once
loop from..to                       # no variable: the count is thrown away
```

The same shape in Rust:

```
for variable in from..=to {}                    // both ends included
for variable in (from..=to).step_by(amount) {}  // skip by a custom amount
for variable in (1..=10).rev() {}               // negative step counts DOWN
// several ranges and values: chain them:
for variable in (1..=3).chain(7..8).chain(19..=20) {}
for _ in from..=to {}                           // the count is thrown away
```

How to use it:

```
fn main()
    # Count up
    loop i 0..3
        put i                # prints 0, 1, 2, 3
    end loop
    # Count by twos
    loop i 0..10 step 2
        put i                # prints 0, 2, 4, 6, 8, 10
    end loop
    # Count down
    loop i 5..1 step -1
        put i                # prints 5, 4, 3, 2, 1
    end loop
    # Mix ranges and single numbers (handy for weird test cases)
    loop n 1..3, 7, 19..20
        put n                # prints 1, 2, 3, 7, 19, 20
    end loop
    # Just want to do something 4 times? Skip the variable.
    loop 0..3
```

```
        put "again"
    end loop
end fn
```

Output:

```
0
1
2
3
0
2
4
6
8
10
5
4
3
2
1
1
2
3
7
19
20
again
again
again
again
```

The same thing in Rust:

```
fn main() {
    for i in 0..=3 {
        println!("{}", i);
    }
    for i in (0..=10).step_by(2) {
        println!("{}", i);
    }
    for i in (1..=5).rev() {
        println!("{}", i);
    }
    for n in [1, 2, 3, 7, 19, 20] {
        println!("{}", n);
    }
    for _ in 0..=3 {
        println!("again");
    }
}
```

Good to know:

- The start of a range can be any expression: `loop i start..total - 1`.
- A literal step of `0` is a compile-time error (the loop could never end).
- `step` is a reserved word — you cannot name a variable `step`.

The collection `loop` — visit every item

What it does: Hands you each item of a list, one at a time.

Syntax:

```
loop item in collection
    # runs once per item
end loop
loop (position, item) in collection.enumerate()
    # runs once per item, also knowing its position (starts at 0)
end loop
```

The same shape in Rust:

```
for item in &collection {
    // runs once per item
}
for (position, item) in collection.iter().enumerate() {
    // runs once per item, also knowing its position (starts at 0)
}
```

How to use it:

```
fn main()
    var fruits := ["apple", "banana", "cherry"]
    loop fruit in fruits
        put fruit
    end loop
    loop (index, fruit) in fruits.enumerate()
        put index.to_string() + ": " + fruit
    end loop
end fn
```

Output:

```
apple
banana
cherry
0: apple
1: banana
2: cherry
```

The same thing in Rust:

```
fn main() {
    let fruits = vec!["apple", "banana", "cherry"];
    for fruit in &fruits {
        println!("{}", fruit);
    }
    for (index, fruit) in fruits.iter().enumerate() {
        println!("{}: {}", index, fruit);
    }
}
```

`break` and `continue` — the loop's steering wheel

Command	What it does	Rust
`break`	Leave the loop immediately.	`break;`
`continue`	Skip the rest of this round; start the next.	`continue;`

How to use it:

```
fn main()
    loop n 1..10
        if n mod 2 = 0
            continue        # skip even numbers
        end if
        if n > 7
            break           # stop before we get tired
        end if
        put n               # prints 1, 3, 5, 7
    end loop
end fn
```

Output:

```
1
3
5
7
```

The same thing in Rust:

```
fn main() {
    for n in 1..=10 {
        if n % 2 == 0 {
            continue;
        }
        if n > 7 {
            break;
        }
        println!("{}", n);
    }
```

```
}
```

One loop to rule them all — a real program

```
fn main()
    put "Multiplication table, rows 1 to 3"
    loop i 1..3
        var row ustring := ""
        loop j 1..3
            row := row + (i * j).to_string().pad_left(4)
        end loop
        put row
    end loop
end fn
```

Output:

```
Multiplication table, rows 1 to 3
   1   2   3
   2   4   6
   3   6   9
```

The same thing in Rust:

```
fn main() {
    println!("Multiplication table, rows 1 to 3");
    for i in 1..=3 {
        let mut row = String::new();
        for j in 1..=3 {
            row.push_str(&format!("{:>4}", i * j));
        }
        println!("{}", row);
    }
}
```

Chapter 9: Functions — your own reusable recipes

What it does: A function is a recipe. You write the steps once, give the recipe a name, and then you can "cook it" (call it) as many times as you like.

Syntax:

```
fn name(parameter: Type, parameter: Type): ReturnType
    # commands
    return value        # hand a result back to whoever called you
end fn
fn name_without_result()
    # commands — no return needed
end fn
```

The same shape in Rust:

```
fn name(parameter: Type, parameter: Type) -> ReturnType {
    // commands
    return value; // hand a result back to whoever called you
}
fn name_without_result() {
    // commands — no return needed
}
```

How to use it:

```
fn add(a: i32, b: i32): i32
    return a + b
end fn
fn say_hello(name: ustring)
    put "Hello, " + name + "!"
end fn
fn main()
    var total i32 := add(2, 3)      # call the recipe, catch the answer
    put total                       # 5
    say_hello("Mia")                # a recipe with no answer to catch
    put add(10, 20)                 # use the answer right away
end fn
```

Output:

```
5
Hello, Mia!
30
```

The same thing in Rust:

```
fn add(a: i32, b: i32) -> i32 {
```

```
    a + b   // Rust lets you drop the `return` for the last line
}
fn say_hello(name: &str) {
    println!("Hello, {}!", name);
}
fn main() {
    let total: i32 = add(2, 3);
    println!("{}", total);
    say_hello("Mia");
    println!("{}", add(10, 20));
}
```

Rules of the road:

The return type goes after a **colon**: `fn add(...): i32`. Rust uses an arrow instead: `fn add(...) -> i32`.

- Close every function with `end fn`.
- No return type means "gives back nothing" — just like `fn greet()`.
- Calling a function is its name and parentheses: `add(2, 3)`.

Functions may be declared inside other functions — `main` can use its own helpers.

Functions that decide something

```
fn is_even(n: i32): bool
    if n mod 2 = 0
        return true
    end if
    return false
end fn
fn main()
    if is_even(4)
        put "4 is even"
    end if
    if not is_even(7)
        put "7 is odd"
    end if
end fn
```

Output:

```
4 is even
7 is odd
```

The same thing in Rust:

```
fn is_even(n: i32) -> bool {
    if n % 2 == 0 {
        return true;
    }
    false
```

```
}
fn main() {
    if is_even(4) {
        println!("4 is even");
    }
    if !is_even(7) {
        println!("7 is odd");
    }
}
```

Chapter 10: Strings — working with text

What it does: Poly's text type is mostly `ustring` (Unicode string). Text can be glued, measured, converted, and cleaned up with helper methods.

Gluing strings with +

```
fn main()
    var first ustring := "Rock"
    var second ustring := "et"
    var joined ustring := first + second      # value position: store the result
    put joined                                # Rocket
    put "Launch " + joined + "!"              # put position: print the glue
end fn
```

Output:

```
Rocket
Launch Rocket!
```

The same thing in Rust:

```
fn main() {
    let first: String = String::from("Rock");
    let second: String = String::from("et");
    let joined: String = format!("{}{}", first, second);
    println!("{}", joined);
    println!("Launch {}!", joined);
}
```

Measuring with `.len()`

Syntax: `text.len()` — the number of **bytes** in the string.

```
fn main()
    var name ustring := "Poly"
    put name.len()          # 4
end fn
```

Output:

```
4
```

The same thing in Rust:

```
fn main() {
    let name: String = String::from("Poly");
    println!("{}", name.len());
```

```
}
```

Turning numbers into text with `.to_string()`

In `put` position Poly glues numbers for you, but when you *store* a glued string in a variable, convert numbers first:

```
fn main()
    var lives i32 := 3
    put "Lives: " + lives                 # fine: put glues automatically
    var label ustring := "Lives: " + lives.to_string()   # store: convert first
    put label
end fn
```

Output:

```
Lives: 3
Lives: 3
```

The same thing in Rust:

```
fn main() {
    let lives: i32 = 3;
    println!("Lives: {}", lives);
    let label: String = format!("Lives: {}", lives);
    println!("{}", label);
}
```

String methods you will use

Method	Example	What it gives you
`.len()`	`"hi".len()`	length in bytes
`.to_string()`	`42.to_string()`	a number turned into text
`.trim()`	`" hi ".trim()`	text without spaces on the ends
`.to_uppercase()`	`"hi".to_uppercase()`	`"HI"`
`.to_lowercase()`	`"HI".to_lowercase()`	`"hi"`
`.contains(x)`	`"a@b".contains("@")`	does it include x? (true/false)
`.starts_with(x)`	`"hi".starts_with("h")`	does it start with x?
`.replace(a, b)`	`"cat".replace("a", "o")`	swap a for b
`.pad_left(n)`	`"7".pad_left(3)`	pad on the left to width n
`.repeat(n)`	`"ab".repeat(3)`	`"ababab"`

How to use them:

```
fn main()
    var shout ustring := "  hello world  "
    put shout.trim().to_uppercase()              # HELLO WORLD
    put shout.trim().replace("world", "Poly")    # hello Poly
    put shout.trim().len()                       # 11
    put "*".repeat(5)                            # *****
end fn
```

Output:

```
HELLO WORLD
hello Poly
11
*****
```

The same thing in Rust:

```
fn main() {
    let shout: String = String::from("  hello world  ");
    println!("{}", shout.trim().to_uppercase());
    println!("{}", shout.trim().replace("world", "Poly"));
    println!("{}", shout.trim().len());
    println!("{}", "*".repeat(5));
}
```

> ***Note:*** `.to_string()` *on numbers works on the Rust and C targets. String + and interpolation work on* ***all*** *targets (Rust, C, asm, JS).*

Characters

```
var letter char := unicode 'A'    # one character, with the unicode keyword
var cafe := unicode "café"        # text with accents works too
```

The same shape in Rust:

```
let letter: char = 'A'; // one character
let cafe = "café";      // text with accents works too
```

The same thing in Rust:

```
let letter: char = 'A';
let cafe: String = String::from("café");
```

Chapter 11: Lists and tuples — many values in one box

Lists (vectors)

What it does: A list (a `Vec` in grown-up talk) holds many values in order, and you can add, remove, and look them up.

Syntax:

```
var numbers := [1, 2, 3]              # a list, type guessed
var names Vec<ustring> := ["Ann", "Bo"]   # a list with the type spelled out
numbers.push(4)                       # add to the end
numbers.len()                         # how many items?
numbers[0]                            # first item (counting starts at 0)
```

The same shape in Rust:

```
let mut numbers = vec![1, 2, 3];                 // a list, type guessed
let mut names: Vec<String> = vec!["Ann", "Bo"];  // a list, type spelled out
numbers.push(4);                                 // add to the end
numbers.len()                                    // how many items?
numbers[0]                                    // first item (starts at 0)
```

How to use it:

```
fn main()
    var scores := [10, 20, 30]
    put scores
    put scores.len()      # 3
    put scores[0]         # 10 — position 0 is the first one
    scores.push(40)       # add one more
    put scores            # [10, 20, 30, 40]
    loop score in scores
        put score
    end loop
end fn
```

Output:

```
[10, 20, 30]
3
10
[10, 20, 30, 40]
10
20
30
40
```

The same thing in Rust:

```
fn main() {
    let mut scores = vec![10, 20, 30];
    println!("{:?}", scores);
    println!("{}", scores.len());
    println!("{}", scores[0]);
    scores.push(40);
    println!("{:?}", scores);
    for score in &scores {
        println!("{}", score);
    }
}
```

Good to know:

- Positions start at **0**: the first item is `[0]`, the second is `[1]`.
- `put` of a whole list prints it like `[10, 20, 30]`.
- Other handy list moves: `pop()` removes and gives back the last item.

  Lists and other collection types work best on the Rust target; for C, keep it simple or use a #c helper (Chapter 18).

Tuples — a tiny fixed grab bag

What it does: A tuple packs a few values together, each with its own type, and you reach each one by its position: `.0`, `.1`, and so on.

How to use it:

```
fn main()
    var pair := (7, "lucky")
    put pair.0     # 7
    put pair.1     # lucky
    var nested := ((1, 2), 3)
    put nested.1       # 3
    put nested.0.1     # 2
end fn
```

Output:

```
7
lucky
3
2
```

The same thing in Rust:

```
fn main() {
    let pair = (7, "lucky");
    println!("{}", pair.0);
    println!("{}", pair.1);
    let nested = ((1, 2), 3);
```

```
    println!("{}", nested.1);
    println!("{}", nested.0.1);
}
```

Tuples often come back from functions that need to return two things at once:

```
fn min_and_max(values: Vec<i32>): (i32, i32)
    # ... find them ...
    return (1, 99)
end fn
```

The same shape in Rust:

```
fn min_and_max(values: Vec<i32>) -> (i32, i32) {
    // ... find them ...
    return (1, 99);
}
```

The same thing in Rust:

```
fn min_and_max(values: &Vec<i32>) -> (i32, i32) {
    // ... find them ...
    (1, 99)
}
```

Chapter 12: Structs — making your own box shapes

What it does: A struct bundles several values under one name, so a "player" can carry a name, a score, and lives all together.

Syntax:

```
struct Name
    field: Type
    field: Type
end struct
# Build one:
var p := Name { field: value, field: value }
# Read a field:
p.field
```

The same shape in Rust:

```
struct Name {
    field: Type,
    field: Type,
}
// Build one:
let p = Name { field: value, field: value };
// Read a field:
p.field
```

How to use it:

```
struct Point
    x: f64
    y: f64
end struct
fn main()
    var p := Point { x: 3.0, y: 4.0 }
    put p.x        # 3
    put p.y        # 4
    put p.x + p.y  # 7
end fn
```

Output:

```
3
4
7
```

The same thing in Rust:

```
struct Point {
```

```
    x: f64,
    y: f64,
}
fn main() {
    let p = Point { x: 3.0, y: 4.0 };
    println!("{}", p.x);
    println!("{}", p.y);
    println!("{}", p.x + p.y);
}
```

Giving a struct jobs with `impl`

What it does: `impl` (short for "implement") attaches functions — called **methods** — to your struct. You then call them with a dot: `thing.do_job()`.

Syntax:

```
impl StructName
    fn method(self): ReturnType
        # use self.field to touch the struct's fields
        return self
    end fn
end impl
```

The same shape in Rust:

```
impl StructName {
    fn method(&self) -> ReturnType {
        // use self.field to touch the struct's fields
        self
    }
}
```

How to use it:

```
struct Counter
    n: i32
end struct
impl Counter
    fn bump(self): Counter
        self.n := self.n + 1
        return self
    end fn
end impl
fn main()
    var c := Counter { n: 0 }
    c := c.bump()     # methods that change things RETURN the new self
    c := c.bump()
    put c.n           # 2
end fn
```

Output:

```
2
```

The same thing in Rust:

```
struct Counter {
    n: i32,
}
impl Counter {
    fn bump(mut self) -> Counter {
        self.n = self.n + 1;
        self
    }
}
fn main() {
    let mut c = Counter { n: 0 };
    c = c.bump();   // in Rust, &mut self would let you skip the re-assign
    c = c.bump();
    println!("{}", c.n);
}
```

The move rule (important!):

Poly **moves** values into calls — like handing over your whole lunchbox. So a method that changes the struct must **return itself**, and you must put it back in the variable:

```
c := c.bump()      # right: catch the returned struct
c.bump()           # wrong: the changed struct is thrown away!
```

The same shape in Rust:

```
c = c.bump(); // right: catch the returned struct
c.bump();     // wrong: the changed struct is thrown away!
```

Also remember: `spawn` and `step` are reserved words, so a method cannot be called `spawn` or `step`.

Chapter 13: Enums and `match` — choices with style

Enums

What it does: An enum defines a **choice**: one of several possible shapes. Each option is called a **variant**, and a variant may carry data.

Syntax:

```
enum Name
    VariantA
    VariantB(Type)
    VariantC(field: Type)
end enum
# Build a variant with ::
var v := Name::VariantB(value)
```

The same shape in Rust:

```
enum Name {
    VariantA,
    VariantB(Type),
    VariantC { field: Type },
}
// Build a variant with ::
let v = Name::VariantB(value);
```

How to use it:

```
enum Direction
    North
    South
    East
    West
end enum
fn main()
    var heading := Direction::East    # the value holds ONE of the four choices
    match heading
        East, put "The sun rises that way"
        _, put "Some other way"
    end match
end fn
```

Output:

```
The sun rises that way
```

(That `match` in the middle is the next section — hold on just a page!)

The same thing in Rust:

```
#[derive(Debug)]
enum Direction {
    North,
    South,
    East,
    West,
}
fn main() {
    let heading = Direction::East;
    match heading {
        Direction::East => println!("The sun rises that way"),
        _ => println!("Some other way"),
    }
}
```

Variants that carry data

```
# Variants can carry values: one number, or two
enum Shape
    Circle(f64)
    Rect(f64, f64)
end enum
fn main()
    var s := Shape::Rect(3.0, 4.0)   # this value carries two numbers inside
    match s
        Rect(w, h), put "a rectangle {w} by {h}"
        _, put "some other shape"
    end match
end fn
```

Output:

```
a rectangle 3 by 4
```

The same thing in Rust:

```
#[derive(Debug)]
enum Shape {
    Circle(f64),
    Rect(f64, f64),
}
fn main() {
    let s = Shape::Rect(3.0, 4.0);
    match s {
        Shape::Rect(w, h) => println!("a rectangle {} by {}", w, h),
        _ => println!("some other shape"),
    }
}
```

The `match` command — pick the right branch

What it does: `match` looks at a value and runs the arm whose pattern fits. It is a stronger, safer `if` chain: every arm starts with a pattern, then a comma, then what to do.

Syntax:

```
match value
    pattern, what to do
    pattern, what to do
    _, what to do          # _ means "anything else"
end match
```

The same shape in Rust:

```
match value {
    pattern => what_to_do,
    pattern => what_to_do,
    _ => what_to_do, // _ means "anything else"
}
```

How to use it (statement form):

```
enum Direction
    North
    South
    East
    West
end enum
fn main()
    var heading := Direction::North
    match heading
        North, put "Up we go!"
        South, put "Down we go!"
        East, put "To the sunrise"
        West, put "To the sunset"
        _, put "No direction"
    end match
end fn
```

Output:

```
Up we go!
```

The same thing in Rust:

```
#[derive(Debug)]
enum Direction {
    North,
    South,
```

```
    East,
    West,
}
fn main() {
    let heading = Direction::North;
    match heading {
        Direction::North => println!("Up we go!"),
        Direction::South => println!("Down we go!"),
        Direction::East => println!("To the sunrise"),
        Direction::West => println!("To the sunset"),
        _ => println!("No direction"),
    }
}
```

Notice: in Poly you write the bare variant name (`North`); Rust wants the full path (`Direction::North`).

Matching on numbers and text

```
fn main()
    var choice i32 := 2
    match choice
        1, put "You picked one"
        2, put "You picked two"
        _, put "Something else"
    end match
end fn
```

Output:

```
You picked two
```

The same thing in Rust:

```
fn main() {
    let choice: i32 = 2;
    match choice {
        1 => println!("You picked one"),
        2 => println!("You picked two"),
        _ => println!("Something else"),
    }
}
```

`match` as an expression — the branch gives you a value

`match` can *produce* a value. You store what the winning arm says. In this form a _ arm is **required**, because every possible input needs an answer.

```
fn main()
    var level i32 := 2
```

```
    var label := match level
        0, "off"
        1, "low"
        _, "high"
    end match
    put label        # high
end fn
```

Output:

```
high
```

The same thing in Rust:

```
fn main() {
    let level: i32 = 2;
    let label = match level {
        0 => "off",
        1 => "low",
        _ => "high",
    };
    println!("{}", label);
}
```

Binding values out of a variant

When a variant carries data, a match arm can grab it with a name:

```
enum Shape
    Circle(f64)
    Rect(f64, f64)
end enum
fn main()
    var s := Shape::Rect(3.0, 4.0)
    match s
        Circle(r), put "circle radius " + r.to_string()
        Rect(w, h), put "rectangle area " + (w * h).to_string()
        _, put "unknown shape"
    end match
end fn
```

Output:

```
rectangle area 12
```

The same thing in Rust:

```
#[derive(Debug)]
enum Shape {
    Circle(f64),
```

```
    Rect(f64, f64),
}
fn main() {
    let s = Shape::Rect(3.0, 4.0);
    match s {
        Shape::Circle(r) => println!("circle radius {}", r),
        Shape::Rect(w, h) => println!("rectangle area {}", w * h),
        _ => println!("unknown shape"),
    }
}
```

Two match forms, one rule of thumb:

- **Statement form** — arms run commands; good inside `main` and loops.
- **Expression form** — arms return a value; a _ arm is mandatory.

Chapter 14: Errors — Result, Ok, Error, and `try`

Why not just crash?

Grown-up programs expect things to go wrong (files hide, networks sleep). Poly's plan: functions that can fail return a **`Result`** — a little box that is either `Ok(value)` (you got something) or `Error(reason)` (you got a problem).

```
# What Result really looks like inside:
enum Result<T, E>
    Ok(T)
    Error(E)
end enum
```

The same shape in Rust:

```
// What Result really looks like inside:
enum Result<T, E> {
    Ok(T),
    Err(E), // Poly spells this variant Error
}
```

> ***Poly vs Rust spelling:*** *Poly uses* `Error(...)`*. Rust calls the same variant* `Err(...)`*. Keep that in your pocket for the Rust column below.*

Handling a Result with `match`

How to use it:

```
fn divide(a: f64, b: f64): Result<f64, ustring>
    if b = 0.0
        return Error("Division by zero")     # the sad path
    end if
    return Ok(a / b)                         # the happy path
end fn
fn main()
    match divide(10.0, 2.0)
        Ok(answer), put "Answer: " + answer.to_string()
        Error(message), error message
    end match
    match divide(10.0, 0.0)
        Ok(answer), put "Answer: " + answer.to_string()
        Error(message), error message
    end match
end fn
```

Output (the second line goes to stderr):

```
Answer: 5
[ERROR] Division by zero
```

The same thing in Rust:

```
fn divide(a: f64, b: f64) -> Result<f64, String> {
    if b == 0.0 {
        return Err(String::from("Division by zero"));
    }
    Ok(a / b)
}
fn main() {
    match divide(10.0, 2.0) {
        Ok(answer) => println!("Answer: {}", answer),
        Err(message) => eprintln!("[ERROR] {}", message),
    }
    match divide(10.0, 0.0) {
        Ok(answer) => println!("Answer: {}", answer),
        Err(message) => eprintln!("[ERROR] {}", message),
    }
}
```

Your own error types

An enum of error variants documents exactly how things can fail, and can carry details (like the limit that was broken):

```
enum ValidationError
    EmptyInput
    TooShort(min: i32)
    TooLong(max: i32)
end enum
fn check_name(name: ustring): Result<ustring, ValidationError>
    if name.len() = 0
        return Error(ValidationError::EmptyInput)
    end if
    if name.len() < 2
        return Error(ValidationError::TooShort(2))
    end if
    return Ok(name)
end fn
fn main()
    match check_name("Al")
        Ok(valid), put "Valid: " + valid
        Error(EmptyInput), error "Name cannot be empty"
        Error(TooShort(min)), error "Name too short, minimum " + min.to_string()
        Error(TooLong(max)), error "Name too long, maximum " + max.to_string()
    end match
end fn
```

Output:

```
Valid: Al
```

The same thing in Rust:

```
#[derive(Debug)]
enum ValidationError {
    EmptyInput,
    TooShort { min: i32 },
    TooLong { max: i32 },
}
fn check_name(name: &str) -> Result<String, ValidationError> {
    if name.len() == 0 {
        return Err(ValidationError::EmptyInput);
    }
    if name.len() < 2 {
        return Err(ValidationError::TooShort { min: 2 });
    }
    Ok(name.to_string())
}
fn main() {
    match check_name("Al") {
        Ok(valid) => println!("Valid: {}", valid),
        Err(ValidationError::EmptyInput) => {
            eprintln!("[ERROR] Name cannot be empty")
        }
        Err(ValidationError::TooShort { min }) => {
            eprintln!("[ERROR] Name too short, minimum {}", min)
        }
        Err(ValidationError::TooLong { max }) => {
            eprintln!("[ERROR] Name too long, maximum {}", max)
        }
    }
}
```

`try` — pass the problem upstairs (without all the matching)

If every function just forwards failures, writing `match` in each one gets tiring. `try` says: "if this is an Error, return it right now; otherwise give me the Ok value."

Syntax:

```
var value := try function_that_returns_a_result()
```

The same shape in Rust:

```
let value = function_that_returns_a_result()?;
```

How to use it:

```
fn load_settings(): Result<ustring, ValidationError>
```

```
    # If check_name fails, try returns the Error from load_settings
    # immediately; otherwise `safe` holds the Ok value.
    var safe := try check_name(unicode "config")
    return Ok(safe)
end fn
```

The same shape in Rust:

```
fn load_settings() -> Result<String, ValidationError> {
    // If check_name fails, ? returns the Err from load_settings
    // immediately; otherwise `safe` holds the Ok value.
    let safe = check_name("config".to_string())?;
    Ok(safe)
}
```

The same thing in Rust:

```
fn load_settings() -> Result<String, ValidationError> {
    let safe = check_name("config")?;   // Rust's `?` is Poly's `try`
    Ok(safe)
}
```

The catch-all arm:

```
match check_name(input)
    Ok(name), put "Valid: " + name
    Error(_), error "Validation failed"    # _ catches every error variant
end match
```

The same shape in Rust:

```
match check_name(input) {
    Ok(name) => println!("Valid: {}", name),
    Err(_) => eprintln!("[ERROR] Validation failed"), // _ catches every error
}
```

The same thing in Rust:

```
match check_name(&input) {
    Ok(name) => println!("Valid: {}", name),
    Err(_) => eprintln!("[ERROR] Validation failed"),
}
```

Chapter 15: Closures — functions you can carry around

What it does: A closure is a small unnamed function you can keep in a variable, hand to other functions, and call later. It can even *remember* variables from the place where it was born.

Syntax:

```
var f := |parameter: Type| expression     # typed parameter
var g := |parameter| expression           # Poly guesses the type
```

The same shape in Rust:

```
let f = |parameter: Type| expression; // typed parameter
let g = |parameter| expression;       // Rust guesses the type
```

How to use it:

```
fn main()
    # A closure with a typed parameter, kept in a variable
    var square := |x: i32| x * x
    put square(6)          # 36
    # A closure that remembers `factor` from around it
    var factor := 10
    var add_factor := |n| n + factor
    put add_factor(5)      # 15
    # You can also write one right where it is needed
    put (|a: i32, b: i32| a + b)(3, 4)     # 7
end fn
```

Output:

```
36
15
7
```

The same thing in Rust:

```
fn main() {
    let square = |x: i32| x * x;
    println!("{}", square(6));
    let factor = 10;
    let add_factor = |n: i32| n + factor;
    println!("{}", add_factor(5));
    println!("{}", (|a: i32, b: i32| a + b)(3, 4));
}
```

Higher-order functions — functions that take functions

What it does: Because functions and closures are values, a function can accept one as a parameter. That is called "higher-order," and it is how you write "do this, but with MY rule."

Syntax:

```
fn name(f: |parameter: Type| ReturnType, value: Type): ReturnType
    return f(value)
end fn
```

The same shape in Rust:

```
fn name(f: impl Fn(Type) -> ReturnType, value: Type) -> ReturnType {
    f(value)
}
```

How to use it:

```
fn apply(f: |x: i32| i32, v: i32): i32
    return f(v)
end fn
fn make_adder(n: i32): |x: i32| i32
    return |x| x + n
end fn
fn main()
    put apply(|x| x * 2, 21)        # 42 — pass a rule in
    var add100 := make_adder(100)   # get a customized rule back
    put add100(5)                   # 105
end fn
```

Output:

```
42
105
```

The same thing in Rust:

```
fn apply(f: fn(i32) -> i32, v: i32) -> i32 {
    f(v)
}
fn make_adder(n: i32) -> impl Fn(i32) -> i32 {
    move |x| x + n
}
fn main() {
    println!("{}", apply(|x| x * 2, 21));
    let add100 = make_adder(100);
    println!("{}", add100(5));
}
```

Vectors also speak this language — they have `map`, `filter`, and friends on the Rust target for turning one list into another.

Chapter 16: Traits and generics — same recipe, many ingredients

Traits (the contract)

What it does: A trait is a promise: "anything with this trait MUST be able to do these jobs." Then `impl Trait for Type` shows how one particular type keeps the promise.

Syntax:

```
trait Name
    fn job(self): ReturnType
end trait
impl Name for Type
    fn job(self): ReturnType
        # how THIS type does the job
    end fn
end impl
```

The same shape in Rust:

```
trait Name {
    fn job(&self) -> ReturnType;
}
impl Name for Type {
    fn job(&self) -> ReturnType {
        // how THIS type does the job
    }
}
```

How to use it:

```
trait Greetable
    fn greet(self): ustring
end trait
struct Person
    name: ustring
    age: i32
end struct
impl Greetable for Person
    fn greet(self): ustring
        return "Hi, I am " + self.name
    end fn
end impl
fn main()
    var p := Person { name: "Ada", age: 30 }
    put p.greet()
end fn
```

Output:

```
Hi, I am Ada
```

The same thing in Rust:

```
trait Greetable {
    fn greet(self) -> String;
}
struct Person {
    name: String,
    age: i32,
}
impl Greetable for Person {
    fn greet(self) -> String {
        format!("Hi, I am {}", self.name)
    }
}
fn main() {
    let p = Person { name: String::from("Ada"), age: 30 };
    println!("{}", p.greet());
}
```

> ***Honest note:*** *traits are part of the Rust-oriented side of Poly. Put anything fancy (trait bounds, default methods) in a* `#rust` *block if a target complains.*

Generics (<T>) — one recipe, any ingredient

What it does: <T> means "any type." Write the function once, and it works for numbers, text, or anything else.

How to use it:

```
fn identity<T>(value: T): T
    return value
end fn
struct Pair<T>
    first: T
    second: T
end struct
fn main()
    var n i32 := identity(42)         # T becomes i32
    var s ustring := identity("hi")   # T becomes ustring
    put n
    put s
    var p := Pair { first: 3, second: 4 }
    put p.first
    put p.second
end fn
```

Output:

```
42
```

```
hi
3
4
```

The same thing in Rust:

```
fn identity<T>(value: T) -> T {
    value
}
struct Pair<T> {
    first: T,
    second: T,
}
fn main() {
    let n: i32 = identity(42);
    let s: String = identity("hi");
    println!("{}", n);
    println!("{}", s);
    let p = Pair { first: 3, second: 4 };
    println!("{}", p.first);
    println!("{}", p.second);
}
```

Chapter 17: Macros, modules, and other organizers

Macros — copy-and-paste power

What it does: A macro is a snippet with holes in it. At parse time Poly copies the body into your code and fills the holes with your arguments. Great for tiny rituals you do over and over.

Syntax:

```
macro name(arguments)
    # body with the argument names used as placeholders
end macro
```

The same shape in Rust:

```
// Rust macros are written with braces (not `end macro`) and are called
// with a ! after the name, like println!("hi"):
macro_rules! name {
    (arguments) => {
        // body with the argument names used as placeholders
    };
}
```

How to use it:

```
macro swap_values(a, b)
    var temp := a
    a := b
    b := temp
end macro
macro shout(text)
    put text.to_uppercase()
end macro
fn main()
    var x i32 := 1
    var y i32 := 2
    swap_values(x, y)      # pastes the swap body with x and y filled in
    put x                  # 2
    put y                  # 1
    shout("hello")         # HELLO
end fn
```

Output:

```
2
1
HELLO
```

The closest Rust (a macro_rules! definition):

```
macro_rules! swap_values {
    ($a:expr, $b:expr) => {{
        let temp = $a;
        $a = $b;
        $b = temp;
    }};
}
macro_rules! shout {
    ($text:expr) => {
        println!("{}", $text.to_uppercase());
    };
}
fn main() {
    let mut x: i32 = 1;
    let mut y: i32 = 2;
    swap_values!(x, y);
    println!("{}", x);
    println!("{}", y);
    shout!("hello");
}
```

Poly macros are simpler than Rust macros: plain names in, code pasted out.

Modules — rooms in your house

What it does: A `module` groups related declarations under one name so your file stays tidy.

Syntax:

```
module name
    # constants, functions, types...
end module
```

The same shape in Rust:

```
mod name {
    // constants, functions, types...
}
```

How to use it:

```
module settings
    const MAX_SCORE := 9999
    const PLAYER_NAME := "Hero"
end module
fn main()
    put settings::MAX_SCORE     # :: reaches into the module
end fn
```

The same shape in Rust:

```
mod settings {
    pub const MAX_SCORE: i32 = 9999;
    pub const PLAYER_NAME: &str = "Hero";
}
fn main() {
    println!("{}", settings::MAX_SCORE); // :: reaches into the module
}
```

The same thing in Rust:

```
mod settings {
    pub const MAX_SCORE: i32 = 9999;
    pub const PLAYER_NAME: &str = "Hero";
}
fn main() {
    println!("{}", settings::MAX_SCORE);
}
```

`use` — import someone else's toolbox

Syntax:

```
use std::collections::HashMap
```

The same shape in Rust:

```
use std::collections::HashMap;
```

The same thing in Rust (it is passed through to Rust unchanged):

```
use std::collections::HashMap;
```

`use` is meant for the Rust target: it lands at the top of the generated Rust file, where the real compiler understands it.

`type` — giving a type a nickname

Syntax:

```
type Miles := i32        # Miles is now just another name for i32
```

The same shape in Rust:

```
type Miles = i32; // Miles is now just another name for i32
```

The same thing in Rust:

```
type Miles = i32;
```

> ***Spot the difference:*** *Poly's* `type` *(and* `let`*) use* `:=`*; Rust's* `type` *uses* `=`*. Same rule as always — Poly's* `:=` *stores.*

pub — the visibility keyword

You may write `pub` before a declaration (`pub fn ...`), but in the current Poly it changes nothing: everything in the file can see everything else.

Chapter 18: Foreign blocks — slipping in real Rust (or C, asm, JS)

What it does: When you need something Poly cannot do — a clever algorithm, a game library, a hardware trick — you open a **foreign block** and write the target language directly. Poly copies it into the generated file exactly as typed.

Syntax:

```
#rust
// real Rust code here
#endrust
#c
/* real C code here */
#endc
#asm
; real assembly here
#endasm
#js
// real JavaScript here
#endjs
```

The same shape in Rust:

```
// #rust ... #endrust: those lines are pasted into the generated Rust
// file exactly as you wrote them. C, asm, and js blocks are handed to
// those languages' compilers instead — Rust never sees them.
```

How to use it:

```
#rust
fn double_value(value: i32) -> i32 {
    value * 2
}
#endrust
fn main()
    var result i32 := double_value(21)   # call the Rust you pasted in
    put result                           # 42
end fn
```

Output:

```
42
```

What Poly generates (the Rust block is copied in, untouched):

```
fn main() {
    let mut result: i32 = double_value(21);
    println!("{}", result);
}
```

```
fn double_value(value: i32) -> i32 {
    value * 2
}
```

The rules of foreign blocks

They must sit at the **top level** of the file — never inside `fn`, `loop`, or any other Poly block. The parser will stop you.

- `#rust` and `#endrust` each get a line to themselves.

  Inside the block, everything belongs to the target language. Poly does not read it; the target's compiler checks it (`rustc` for `#rust`, `cc` for `#c`, and so on).

  No running commands in a foreign block — only definitions. Poly's `fn main()` is where the action starts.

  `#c` blocks pair with `--target c`, `#asm` with `--target asm`, `#js` with `--target js`. A source containing `#cpp` is rejected: there is no C++ backend yet.

Telling Poly about a foreign function (optional `extern`)

By default Poly trusts your foreign calls and lets `rustc` check them. If you want Poly to check the spelling too, declare the signature:

```
extern rust fn double_value(value: i32): i32
#rust
fn double_value(value: i32) -> i32 { value * 2 }
#endrust
```

The same shape in Rust:

```
// In Rust there is nothing extra to declare: the #rust block already
// contains the real definition.
fn double_value(value: i32) -> i32 { value * 2 }
```

Rules: top level only, and the target (`rust`, `c`, `asm`, `js`) must match `--target`. Declarations are checked by Poly and never copied into the output. With `--strict`, every foreign call must have one.

Bringing in outside crates with dep

What it does: If your `#rust` block uses an outside library (a "crate"), name it at the top of your file. Poly writes it into the generated `Cargo.toml` for you.

How to use it:

```
dep rand = "0.8"
dep libc = "0.2"
fn main()
    put "these crates are available inside #rust blocks"
```

```
end fn
```

The same shape in Rust:

```
# In Rust, `dep rand = "0.8"` becomes a line in Cargo.toml:
[dependencies]
rand = "0.8"
libc = "0.2"
```

What it becomes — the generated `Cargo.toml`:

```
[dependencies]
rand = "0.8"
libc = "0.2"
```

Good to know:

- Only the Rust target has dependencies; C, asm, and JS warn and ignore `dep`.
- The version must be a plain Cargo version string (`"1"`, `"0.9"`, `"^1.2"`).

Which block for which target?

Block	Paired with	Target language
`#rust` ... `#endrust`	`--target rust` (default)	Rust
`#c` ... `#endc`	`--target c`	C11
`#asm` ... `#endasm`	`--target asm`	x86-64 assembly (Linux)
`#js` ... `#endjs`	`--target js`	JavaScript (ES2020)
`#cpp` ... `#endcpp`	*(reserved)*	rejected until C++ exists

Chapter 19: Async and spawn — waiting without waiting

> *This chapter is a **Rust-target only** feature. The C, assembly, JS, and Goose targets reject async functions and will point you back at foreign blocks.*

What it does: An `async fn` is a function that can pause ("await") while slow work happens, instead of freezing your whole program. `spawn` throws a task into the background and carries straight on.

Syntax:

```
async fn name(params): ReturnType
    # ... slow.start().await    # pause here until it is done
    return value
end fn
spawn expression        # launch a background task, do not wait
```

The same shape in Rust:

```
async fn name(params) -> ReturnType {
    // ... slow.start().await // pause here until it is done
    value
}
tokio::spawn(expression); // launch a background task, do not wait
```

How to use it:

```
async fn compute(x: i32): i32
    delay(10).await            # sleep 10 milliseconds, politely pause
    return x * x
end fn
async fn main()                # an async main is allowed to .await
    var a := compute(3).await  # wait for this one...
    var b := compute(4).await  # ...then this one
    put a + b                  # 25
    spawn compute(5)           # fire-and-forget: do not wait at all
end fn
```

Output:

```
25
```

The same thing in Rust:

```
use std::time::Duration;
async fn compute(x: i32) -> i32 {
    tokio::time::sleep(Duration::from_millis(10)).await;
    x * x
}
```

```
#[tokio::main]
async fn main() {
    let a = compute(3).await;
    let b = compute(4).await;
    println!("{}", a + b);
    tokio::spawn(compute(5));   // background: nobody waits for it
}
```

Good to know:

- Only async functions may use `.await`.
- `delay(ms)` is Poly's built-in sleep.

  `spawn expr` runs `expr` as a background task (Tokio on the Rust target). Nothing collects its result — use it for side effects, like saving a file in the background.

  Awaiting inside a loop runs the tasks **one after another**. To go parallel, `spawn` them instead.
- `spawn` is a reserved word; name your methods `spawn_piece`, not `spawn`.

Chapter 20: The `poly` command — running your programs

`poly` is the command-line tool that reads your `.poly` file and does whatever you ask. This chapter is the full tour.

The everyday commands

```
poly program.poly                # translate + build (Rust target by default)
poly --check program.poly        # check everything WITHOUT keeping output
poly --target js program.poly    # translate to JavaScript and run it with Node
poly --target c program.poly     # translate to C and build a C executable
poly --help                      # show the option list
poly --version                   # show the version
```

When a default build succeeds it tells you where your program went:

```
Generated and built program.poly -> rust_output/program/target/debug/program
```

Run that file to start your program. (The JavaScript target runs it for you right away.)

Choosing a target

```
poly --target rust program.poly    # default: Rust
poly --target c program.poly       # C11
poly --target asm program.poly     # x86-64 assembly (Linux)
poly --target js program.poly      # JavaScript (ES2020, run with Node)
```

Rust is the default, so `--target rust` can be left out.

Looking at what Poly made

```
poly --emit-rust program.poly              # print the generated Rust
poly --target c --emit-c program.poly      # print the generated C
poly --target asm --emit-asm program.poly  # print the generated assembly
poly --target js --emit-js program.poly    # print the generated JavaScript
poly --intermediate-representation program.poly   # print the IR (Rust target)
poly --ir program.poly                     # shorter spelling of the same
poly --source-map program.poly             # print the source map
```

These print code and exit — they never build anything. Perfect for seeing exactly how your `put` turned into a `println!`.

Checking without building

```
poly --check program.poly              # parse + check + transpile + compile
poly --target c --check program.poly   # the same for the C target
```

```
poly --strict program.poly              # demand extern declarations for
                                        # every foreign call
```

`--check` is what you run in a hurry (or in CI): it proves the program is valid, then throws the build away.

Peeking under the hood

```
poly --tokens program.poly    # the words and symbols the lexer found
poly --ast program.poly       # the tree the parser built
```

When the compiler says "unexpected token," `--tokens` shows you exactly what it saw.

Bigger tools

```
poly --project my_game my_game.poly   # generate a whole Cargo project
poly --format program.poly            # print Rust formatted with rustfmt
poly --diff program.poly              # show the diff before/after formatting
poly --watch program.poly             # rebuild every time you save (Rust)
poly --repl                           # talk to Poly interactively
```

`--project` writes a real Cargo project into a folder, including a `.poly-generated` marker so a later rebuild knows it owns that folder. Your `dep` declarations (Chapter 18) are written into the generated `Cargo.toml`.

Flag order does not matter

You may put the file first or the flags first — `poly` understands both:

```
poly --target c --check game.poly
poly game.poly --target c --check
```

A safety warning, seriously

`poly` **runs code**. Building a `.poly` file executes build scripts and compiles whatever is pasted in its foreign blocks. Only run `poly` on files you wrote or fully trust — and treat every `.poly` file in a pull request as a program that wants control of your machine.

Chapter 21: When your program grumbles — common mistakes

Poly's error messages try to be helpful. Here are the classics.

== is rejected

```
Poly uses `=` to compare and `:=` to store.
```

Fix: `if score = 10` and `score := score + 1`.

A block that never ends

Every opener needs its own closer:

You opened	Close it with
`fn`	`end fn`
`if`	`end if`
`while`	`end while`
`loop`	`end loop`
`match`	`end match`
`struct`	`end struct`
`enum`	`end enum`
`trait`	`end trait`
`impl`	`end impl`
`module`	`end module`
`macro`	`end macro`
`unsafe`	`end unsafe`
`#rust`	`#endrust`
`#c`	`#endc`

`var x: i32 := 5` is wrong

Fix: `var x i32 := 5` — `var` types take **no colon**. (`let` types do: `let x: i32 := 5`.)

Retired syntax the compiler will nag about

Old (rejected)	New (correct)
`x += 3`	`x := x + 3`
`set x to y`	`x := y`
`a % b`	`a mod b`
`a && b`, `` `a\ ``	`\`
`var x = 5`	`var x := 5`
`loop i: 0..10`	`loop i 0..10` (no colon)

Reserved words — names you cannot use

`spawn` — launches background tasks (so it cannot be a variable or method name; try `spawn_piece`).

- `step` — introduces a loop's step clause.
- `validate` — belongs to the parser's `with validate` compatibility form.

Depth limits

Poly refuses to build silly-depth trees (on purpose — it keeps hostile files from crashing it):

- statement nesting: **128** levels
- expression nesting: **32** levels
- chained `else if` arms: **32** levels
- one flat operator chain: **512** operands

Fix: flatten the structure — pull deep pieces into functions.

A foreign block inside a function

Foreign blocks (`#rust`, `#c`, ...) must appear at program scope, at the left margin of the file. **Fix:** move it to the top level; Poly code inside `fn main()` can then call its functions.

Type complaints

Poly compares strictly: `u64 = i64` or `1 + 0.5` is an error. **Fix:** convert on purpose with `as`, e.g. `bits as i64 = row`.

Appendix A: The type menu

Poly type	Size	Rust equivalent	What it holds
`bool`	1 byte	`bool`	`true` / `false`
`i8` / `u8`	1 byte	`i8` / `u8`	tiny whole numbers
`i16` / `u16`	2 bytes	`i16` / `u16`	small whole numbers
`i32` / `u32`	4 bytes	`i32` / `u32`	everyday whole numbers
`i64` / `u64`	8 bytes	`i64` / `u64`	big whole numbers
`i128` / `u128`	16 bytes	`i128` / `u128`	enormous numbers (Rust target)
`isize` / `usize`	pointer	`isize` / `usize`	sizes and positions
`f32`	4 bytes	`f32`	decimals, less precise
`f64`	8 bytes	`f64`	decimals, everyday choice
`char` / `uchar`	4 bytes	`char`	one character, like `'A'`
`string`	varies	`String`	byte-oriented text
`ustring`	varies	`String`	Unicode text (your default)
`byte`	1 byte	`u8`	one raw byte
`bytes`	varies	`Vec<u8>`	a buffer of raw bytes
`ptr T`	pointer	`*const T`	a raw memory address

Containers and helpers: `Vec<T>` (list), `Map<K, V>` (key + value), `Set<T>` (unique values), `Box<T>`, `Rc<T>`, `Arc<T>`, `Option<T>` (a value or nothing), `Result<T, E>` (a value or an error), and tuples like `(i32, ustring)`.

`i`, u, `f` — the letters mean: `i` = integer (can be negative), u = unsigned (never negative), `f` = float (decimal). The number is the width in bits: more bits = bigger range.

Appendix B: The operator shelf

Job	Poly writes	Rust writes
Store	`x := 5`	`let mut x = 5;` / `x = 5;`
Compare equal	`x = 5`	`x == 5`
Not equal	`x != 5`	`x != 5`
Add / subtract	`a + b`, `a - b`	same
Multiply / divide	`a * b`, `a / b`	same
Remainder	`a mod b`	`a % b`
And / or / not	`a and b`, `a or b`, `not a`	`a && b`, `` `a` ``
Bitwise and/or	`a bitand b`, `a bitor b`	`a & b`, `` `a` ``
Bitwise xor/not	`a xor b`, `bitnot a`	`a ^ b`, `!a`
Shifts	`a shift left n`, `a << n`	`a << n`
Range	`0..10`, `0..=10`	`0..10`, `0..=10`
Cast	`x as i64`	`x as i64`
Module path	`Enum::Variant`	`Enum::Variant`

Appendix C: Every command at a glance

Every Poly command on this page also has a Rust translation — find them all side by side in Appendix D.

```
# ---- Comments ----
# line comment          // line comment          /* block comment */
# ---- Values ----
var name Type := value      # make a changeable box (no colon on the type)
var name := value           # make a box, type guessed
let name: Type := value     # make an unchangeable box (colon before type)
const NAME := value         # a fixed number for the whole file
name := value               # store a new value
value as Type               # convert a value's type
# ---- Output ----
put expression                       # print + newline (stdout)
put expression to "file"             # write/overwrite a file
put expression to "file" -append     # add to the end of a file
error expression                     # [ERROR] line (stderr)
warn expression                      # [WARN] line (stderr)
info expression                      # [INFO] line (stderr)
# ---- Input ----
var x := get                             # one line from the keyboard
var x := get unicode "Prompt: "          # with a prompt
var x := get --as i32                    # convert to a number
var x := get --default unicode "fallback" # used when input is empty
var x := get --mask unicode "*"          # hidden typing
var x := get --until unicode ","         # stop at a delimiter
var x := get --timeout 3000              # give up after 3000 ms
var x := get from "file"                 # read from a file
var x bytes := get from "file" --bytes 8  # read 8 bytes
# ---- Decisions ----
if condition
    ...
else if other condition
    ...
else
    ...
end if
# ---- Loops ----
while condition ... end while
loop ... end loop                          # forever (use break)
loop i 0..10 ... end loop                  # both ends included
loop i 0..10 step 2 ... end loop           # custom jump
loop i 10..1 step -1 ... end loop          # countdown
loop i 1..3, 7, 19..20 ... end loop        # mix ranges and values
loop 0..10 ... end loop                    # count is thrown away
loop item in list ... end loop             # visit each item
loop (n, item) in list.enumerate() ... end loop   # with positions
break                                      # leave the loop
continue                                   # skip to next round
```

```
# ---- Functions ----
fn name(a: Type, b: Type): ReturnType
    return value
end fn
fn name()                                   # no result
end fn
# ---- Data ----
struct Name ... end struct
var thing := Name { field: value }
impl Name ... end impl                      # methods; use c := c.bump()
enum Name ... end enum
var v := Name::Variant                      # build a variant
match value
    pattern, action
    _, action                               # everything else
end match
# ---- Errors ----
return Ok(value)                            # it worked
return Error(reason)                        # it broke
var x := try risky_function()               # forward failures upstairs
# ---- Functions as values ----
var f := |x: Type| expression               # a tiny unnamed function
fn name(f: |x: Type| R, v: Type): R ... end fn    # take a function
# ---- Organization ----
trait Name ... end trait                    # a contract
impl Trait for Type ... end impl            # keeping the promise
fn name<T>(value: T): T ... end fn          # generic: any type
macro name(args) ... end macro              # copy-paste at parse time
module name ... end module                  # a folder for related things
use path::to::thing                         # borrow from another toolbox
type Nickname := Type                       # a type's second name
# ---- Other languages ----
#rust ... #endrust                          # paste real Rust
#c ... #endc                                # paste real C
#asm ... #endasm                            # paste assembly
#js ... #endjs                              # paste JavaScript
extern rust fn name(a: Type): Type          # describe a foreign function
dep crate_name = "1.2"                      # an outside crate (Rust target)
# ---- Async (Rust target) ----
async fn name(): Type ... end fn            # a pausable function
value.await                                 # pause until finished
spawn expression                            # background task
delay(100).await                            # sleep 100 ms
```

Appendix D: Poly to Rust — the big cheat sheet

Poly	Rust
`var x i32 := 0`	`let mut x: i32 = 0;`
`var x := 0`	`let mut x = 0;`
`let x: i32 := 0`	`let x: i32 = 0;`
`const MAX := 100`	`const MAX: i32 = 100;`
`x := x + 1`	`x = x + 1;`
`x := x + 1` (retired `x += 1`)	`x += 1;`
`put "hello"`	`println!("{}", "hello");`
`put "n = " + n`	`println!("n = {}", n);`
`put "hi {name}"`	`println!("hi {}", name);`
`error "bad"`	`eprintln!("[ERROR] {}", "bad");`
`var s := get`	read a line from stdin
`var n := get --as i32`	parse a line from stdin as `i32`
`if a = b`	`if a == b {`
`while ready`	`while ready {`
`loop i 0..10`	`for i in 0..=10 {`
`loop i 0..10 step 2`	`for i in (0..=10).step_by(2) {`
`loop x in xs`	`for x in &xs {`
`loop ... end loop` (infinite)	`loop {`
`break` / `continue`	`break;` / `continue;`
`fn add(a: i32, b: i32): i32`	`fn add(a: i32, b: i32) -> i32`
`return a + b`	`return a + b;`
`struct Point ... end struct`	`struct Point { ... }`
`impl Counter ... end impl`	`impl Counter { ... }`
`enum Shape ... end enum`	`enum Shape { ... }`
`Shape::Circle(r)`	`Shape::Circle(r)`
`match x ... end match`	`match x { ... }`
`Ok(v)` / `Error(e)`	`Ok(v)` / `Err(e)`
`try expr`	`expr?`

Poly	Rust
`var a := [1, 2]`	`let mut a = vec![1, 2];`
`a.push(3)`	`a.push(3);`
`a.len()`	`a.len()`
`pair.0`	`pair.0`
`` `\ ``	x\
`n.to_string()`	`n.to_string()`
`n as i64`	`n as i64`
`a mod b`	`a % b`
`a and b`	`a && b`
`xor`, `bitand`, `bitor`	`^`, `&`, `` `\ ``
`spawn work()`	`tokio::spawn(work());`
`value.await`	`value.await`
`#rust ... #endrust`	copied into the Rust file untouched
`dep rand = "0.8"`	`rand = "0.8"` in `Cargo.toml`
`end fn`, `end if`, `end loop`	`}`, `}`, `}`

Glossary

- **Boolean** — a `bool`: just `true` or `false`.
- **Bug** — a mistake in your program that makes it misbehave.

  Crate — a box of Rust code written by someone else that you can borrow (Poly declares them with `dep`).

  Closure — a tiny unnamed function you can store in a variable and carry around.
- **Compile** — turn human-readable code into machine code.
- **Constant** — a value that never changes (`const`).
- **Enumerator (enum)** — a type with a fixed list of choices.
- **Expression** — anything that produces a value, like `2 + 3`.

  Foreign block — a section of your file written directly in Rust, C, assembly, JavaScript, or Goose.
- **Function (`fn`)** — a named recipe you can run whenever you like.
- **Immutable** — unable to change (Poly's `let` boxes).
- **Inference** — Poly guessing a type from the value you gave it.
- **Integer** — a whole number without a fractional part.
- **Loop** — running the same commands again and again.
- **Macro** — a code template filled in at parse time.

  Method — a function attached to a struct with `impl`, called with a dot: `thing.bump()`.
- **Module** — a named folder for related declarations (`module`).
- **Mutable** — able to change (Poly's `var` boxes).
- **Null** — a "nothing" value; Poly uses `Option<T>` instead.
- **Parameter** — a named input a function expects.
- **Parse** — reading your text and building the compiler's tree of it.
- **Rust** — the systems language Poly translates to by default.
- **Statement** — one instruction, like `put "hi"`.
- **Struct** — a bundle of named fields you designed yourself.
- **Syntax** — the rules for how commands are spelled.
- **Target** — which language Poly translates to: rust, c, asm, js, or goose.

  Transpiler — a translator between two programming languages (Poly is one).
- **Type** — the kind of value a variable holds, like `i32` or `ustring`.
- **Variable** — a labeled box that holds a value (`var`).

Where to go next

Read the official docs in this repository: `POLY_TUTORIAL.md` (more lessons), `POLY_QUICK_REFERENCE.md` (fast reminders), and `POLY_SPEC_v2.md` (the full contract).

Open the `examples/` folder — every `.poly` file there is a real, working program you can run and pick apart.

Type the examples in this book yourself. Changing one number and re-running is the fastest way to learn.

Happy hacking — and may your `end fn` always be found!